

Application of Neural Networks

Kannan Singaravelu, CQF

June 2023

Contents

1 Deep Sequence Modeling **2**
1.1 Recurrent Neural Network . . . 2
1.2 Long Short Term Memory Network . . . 3

2 TensorFlow = Tensor + Flow **3**
2.1 TensorFlow 2 Installation . . . 4
2.2 Keras . . . 5
2.3 TensorBoard . . . 5

3 Many-to-One Sequence Problem **6**
3.1 Example : Many-to-One Single Feature . . . 7

4 Applying Neural Networks to Financial Time Series **8**
4.1 Path to store results . . . 8
4.2 Retrieve data . . . 8
4.3 Visualization . . . 9
4.4 Train - Test - Split . . . 9
4.5 Preprocessing . . . 9
4.5.1 Generate sequences . . . 9
4.5.2 Reshape . . . 10
4.6 Building a LSTM Model . . . 10
4.6.1 Train the model . . . 11
4.7 Evaluate model performance . . . 12
4.7.1 Prediction . . . 12
4.7.2 Plot results . . . 12

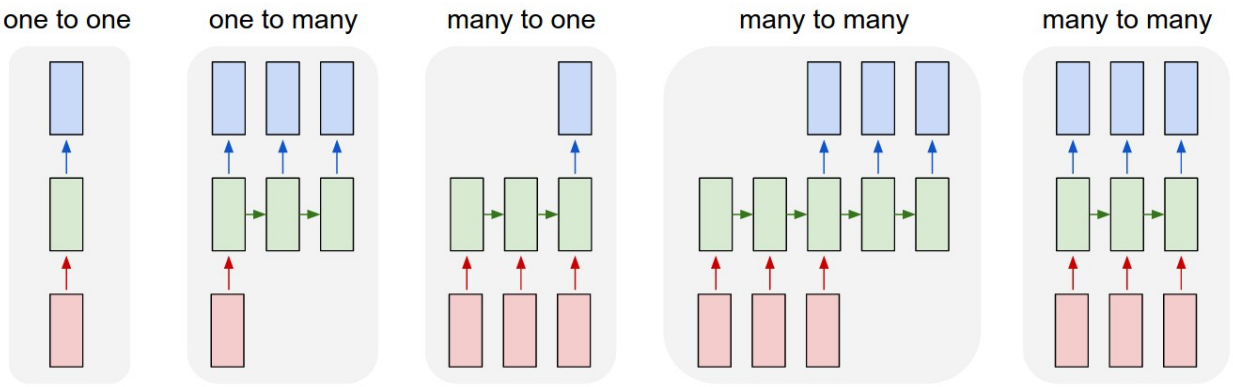
5 References **13**

1 Deep Sequence Modeling

Deep sequence modeling is essentially applying neural network to problems involving sequential processing of data. Sequence data has memory and comes in many forms such as text, audio, video and financial time series. Thus, requiring a different modeling approaches.

1.1 Recurrent Neural Network

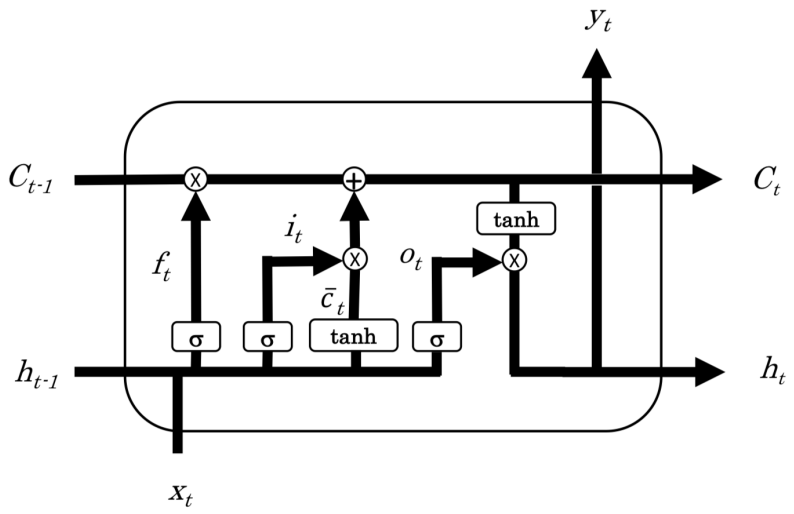
Recurrent Neural Network (RNN) generalize the feedforward neural network with an internal memory state and good at modeling sequence data as it uses sequential memory for prediction. Sequential memory is a mechanism used to identify the sequence patterns. RNNs are faster and use less computational resources as there are less tensor operations. Sequence problems can be broadly categorized into the following types



Diagrammatic Representation of RNNs for Sequence Modeling [1]

1.2 Long Short Term Memory Network

Long Short Term Memory (LSTM) algorithm is a special kind of RNN, explicitly designed to avoid the long-term dependency problem by addressing the problem of short term memory. LSTM models are fundamental to deep learning for financial timeseries and have four interacting layers as given below



$$f_t = \sigma\,(W_f . [h_{t-1}, x_t] + bf)$$

$$i_t = \sigma\,(W_i . [h_{t-1}, x_t] + bi)$$

$$\bar{c}_t = tanh\,(W_c . [h_{t-1}, x_t] + b_c)$$

$$C_t = f_t * Ct_{-1} + it * \bar{c}_t$$

$$o_t = \sigma\,(W_o . [h_{t-1}, x_t] + bo)$$

$$h_t = o_t * tanh\,(C_t)$$

Diagrammatic Representation of LSTM cell and its operation [2]

2 TensorFlow = Tensor + Flow

TensorFlow is an open-source library for numerical computation originally developed by researchers and engineers working at the Google Brain team. It was originally built as a proprietary machine learning system based on deep learning neural networks.

TensorFlow is made up of two words tensor and flow where Tensor is a multidimensional array and Flow is a graph of operations. Internally, TensorFlow implements machine learning algorithms as a graph of operations on multidimensional array. Tensor is a container for data to store numerical data (almost always) and are a generalization of matrices to an arbitrary number of dimensions which are also called as axis or rank. A tensor is defined by three key attributes:

- a) Number of axis or ranks `ndim`
- b) Shape `shape`
- c) Data type `dtype`

TensorFlow is preferred for the following reason

- Easy to build and deploy machine learning models for a beginner.
- Build state of art machine learning models with Keras functional API and model sub classing APIs.
- Supports production of machine learning models, anywhere from CPUs, GPUs to edge devices as well as web servers.
- APIs available for Python, Java and Go programming languages.

- Has a very flexible architecture
- Enables easy deployment across different hardware platforms like CPUs, TPUs and GPUs, and computing devices like desktops, servers, mobile devices, and edge devices.

TensorFlow is a foundation (base) library that can be used to create deep learning models directly or by using wrapper libraries that simplify the process built on top of it. It is an open source (symbolic math) library for fast numerical computing, created and maintained by Google. Like NumPy, the primary purpose of TensorFlow is to manipulate mathematical expressions over numerical tensors. It can run not only on CPU, but also on GPUs and TPUs, highly-parallel hardware accelerators.

Theano is an open source project developed by now Montreal Institute for Learning Algorithms group at the University of Montreal. However, Theano is considered effectively dead with the team announcing on September 2017 that they would cease all its major development after the release of 1.0 version.

2.1 TensorFlow 2 Installation

Refer here for installation of latest version of TensorFlow. Tensorflow is tested and supported on the 64-bit systems and has the following system requirements:

- Python 3.7–3.10
- pip 19.0 (or >20.3 for macOS)
- Ubuntu 16.04 or later
- Windows 7 or later (with C++ redistributable)
- macOS 10.12.6 (Sierra) or later (no GPU support)
- Raspbian 9.0 or later : GPU support requires a CUDA®-enabled card (Ubuntu and Windows)
- WSL2 via Windows 10 19044 or higher including GPUs (Experimental)

Install the Python development environment on your system (recommended)

pip install –user virtualenv pip install –upgrade tensorflow

n!

Alternatively, install the current stable release for CPU and GPU if above requirements were already satisfied,

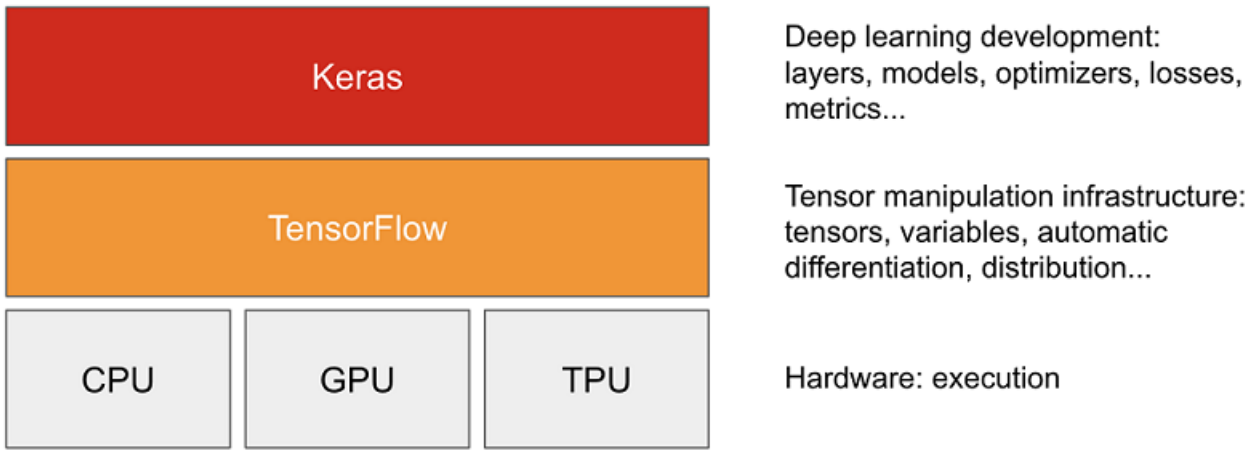
pip install tensorflow

conda install -c anaconda tensorflow

2.2 Keras

Keras is a deep learning API for Python and is one of the most powerful and easy to use library which runs on top of popular libraries like TensorFlow and Theano for creating deep learning models. It was originally created and developed by Google AI Developer/Researcher, Francois Chollet to facilitate his own research and experiement. He committed and released the first version of Keras to his GitHub on March 2015.

Originally, Keras' default backend was Theano. At the same time, Google had released TensorFlow. Given the synergies, Keras started supporting TensorFlow as a backend, and slowly TensorFlow became the most popular backend, resulting in TensorFlow being the default backend starting from the release of Keras v1.1.0.



Deep Learning Software and Harware Stack

The core data structures of Keras are layers and models. The simplest type of model is the Sequential model, which is a linear stack of layers. For more complex architectures, we should use the Keras functional API, which allows to build arbitrary graphs of layers.

Keras is ideal for prototyping simple concepts and comes with a large community support. It is the default backend now shipped with tensorflow. Refer here for Keras API.

2.3 TensorBoard

TensorBoard is a browser-based visualization tool that let you monitor models during training and gives access to the following features

- Visually monitoring metrics during training
- Visualizing your model architecture
- Visualizing histograms of activations and gradients

[ ]:
```
# Create directory
# $ mkdir my_log_dir

# To access http://localhost:6006
# $tensorboard --logdir=mlogs
```

[ ]:
```
# Load the TensorBoard notebook extension
%load_ext tensorboard
```

3 Many-to-One Sequence Problem

> The input of the LSTM is always a 3D array *[batch, timesteps, feature]* The output of the LSTM could be a 2D array or 3D array depending upon the `return_sequences` argument If `return_sequence` is `False`, the output is a 2D array *[batch, feature]* If `return_sequence` is `True`, the output is a 3D array *[batch, timesteps, feature]*

- `batch` is the number of samples in the input data
- `timesteps` are the number of timesteps per sample
- `feature` correspond to the number of features per timestep

Data with single time-step cannot be considered sequence data in a real sense. Densely connected neural networks have been proven to perform better with single time-step data. Real sequence data consists of multiple time-steps, such as stock prices of past n days, a sentence containing multiple words, and so on.

In many-to-one sequence problems, each input sample has more than one timestep, however the output consists of a single element. Similar to the above examples, each timestep in the input can have one or more features.

Imports & Settings

[ ]:
```
# import tensorflow and keras
import tensorflow
```

[ ]:
```
# check the tensorflow and keras version
print(f" TensorFlow version is {tensorflow.__version__} \n Keras version is␣
 ↪{tensorflow.keras.__version__}")
```

[ ]:
```
"""
Created on Wed Oct 27
@author: Kannan Singaravelu
"""

# supress warnings
import warnings
warnings.filterwarnings('ignore')

import os
import random
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

# data manipulation
import numpy as np
import pandas as pd
import datetime
```

```
from pathlib import Path

# scikit-learn modules
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# tensorflow modules
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, LSTM

from tensorflow.keras.optimizers import Adam, RMSprop
from tensorflow.keras.utils import plot_model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint,␣
↪TensorBoard

# plotting & outputs
import matplotlib.pyplot as plt
plt.style.use('seaborn')
from pprint import pprint
```

[ ]:
```
def set_seeds(seed=2022):
    random.seed(seed)
    np.random.seed(seed)
    tensorflow.random.set_seed(seed)
```

3.1 Example : Many-to-One Single Feature

Let's prepare a sample dataset of 20 samples with 3 timesteps where each timestep will consist of a single feature. The output of each sample will be the sum of these 3 inputs. For example, if our sample consists of a sequence 1,2,3, the output would be sum of these inputs, 1+2+3 = 6.

[ ]:
```
# create sample dataset
X = np.array([i+1 for i in range(60)])
```

[ ]:
```
# reshape into 3D
X = np.array(X).reshape(20,3,1)
```

[ ]:
```
print(f'X: {X}')
```

[ ]:
```
# check the shape
X.shape
```

Next, we'll create output y, which is the sum of the values in the timesteps

[ ]:
```
# y is the sum of the values in the timesteps
y = []
for each in X:
```

```
    y.append(each.sum())

# convert to array
y = np.array(y)

# check the output
print(f'y: {y}')
```

[ ]:
```
# check the shape
y.shape
```

[ ]:
```
# compile model five
nn = Sequential()
nn.add(LSTM(50, activation='relu', input_shape=(3, 1)))
nn.add(Dense(1))
nn.compile(optimizer='adam', loss='mse')
print(nn.summary())
```

[ ]:
```
# fit model
nn.fit(X, y, batch_size=5, epochs=2000, validation_split=0.2, verbose=0)
```

[ ]:
```
# predict the outcome
test_input = np.array([70,71,72])
test_input = test_input.reshape((1, 3, 1))
test_output = nn.predict(test_input, verbose=0)
print(test_output)
```

4 Applying Neural Networks to Financial Time Series

Given that we have some understanding of the above deep sequence modeling, let us now apply neural networks to financial timeseries which is a many-to-one sequence problem.

We will retrive the 20 years of SPY price data that we stored earlier for this exercise. We then apply LSTM network given the nature of financial timeseries with a lookback (window size) of 60 days.

4.1 Path to store results

[ ]:
```
results_path = Path('results', 'lstm_time_series')
if not results_path.exists():
    results_path.mkdir(parents=True)
```

4.2 Retrieve data

[ ]:
```
# Read data from the locally stored file
data = pd.read_csv('data/spy.csv', index_col=0, parse_dates=True)[['Adj␣
↪Close']]['2000':'2019']
```

[ ]:
```
# Check the shape - has to be 2D
data.shape
```

[ ]:
```
# Check for missing values
data.isna().sum()
```

4.3 Visualization

[ ]:
```
# Visualization
plt.figure(figsize=(14,6))
plt.title('SPY Price')
plt.plot(data);
```

4.4 Train - Test - Split

[ ]:
```
# Splitting the datasets into training and testing data.
train_data, test_data = train_test_split(data, train_size=0.8, test_size=0.2,␣
 ↪shuffle=False)

# Output the train and test data size
print(f"Train and Test Size {len(train_data)}, {len(test_data)}")
```

4.5 Preprocessing

[ ]:
```
# Scale the features MinMax for training and test datasets
scaler = MinMaxScaler()
scaled_train_data = scaler.fit_transform(train_data)
scaled_test_data = scaler.transform(test_data)
```

4.5.1 Generate sequences

We'll create a function to generate sequence in the format required for deep sequence modeling.

[ ]:
```
def generate_sequence(data, sequence_length=60):

    # create X & y data array
    X = []
    y = []

    for i in range(sequence_length, len(data)):
        X.append(data[i - sequence_length:i, 0])
        y.append(data[i, 0])

    # Converting x_train and y_train to Numpy arrays
    return np.array(X), np.array(y)
```

[ ]:
```
# sequence length
lookback = 60
```

[ ]:
```
X_train, y_train = generate_sequence(data=scaled_train_data,␣
↪sequence_length=lookback)
print(f'X_train: {X_train.shape}, y_train {y_train.shape}')
```

[ ]:
```
X_test, y_test = generate_sequence(data=scaled_test_data,␣
↪sequence_length=lookback)
print(f'X_test: {X_test.shape}, y_test {y_test.shape}')
```

4.5.2 Reshape

We need to reshape the data as LSTM network consume input in a 3-dimensional array in the form of `n_samples x timesteps x n_features`.

[ ]:
```
# reshaping array
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
y_train = y_train[:, np.newaxis]

# check the array size
print(f'X_train Shape: {X_train.shape}, y_train {y_train.shape}')
```

[ ]:
```
# reshaping array
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
y_test = y_test[:, np.newaxis]

# check the array size
print(f'X_test Shape: {X_test.shape}, y_test {y_test.shape}')
```

4.6 Building a LSTM Model

First, we define the LSTM Recurrent Neural Network by initializing the **Sequential** Model. We'll add more **LSTM** layers and adjust the **Dropout** in order to improve the accuracy of our model. Finally, a **Dense** layer is used as output layer.

Next, we compile and train the model defined above. Iteratively, we can increase or decrease the epochs and batch size to get more accuracy.

[ ]:
```
# Create a model
def create_model(hu=256, lookback=60):

    tensorflow.keras.backend.clear_session()

    # instantiate the model
    model = Sequential()
    model.add(LSTM(units=hu, input_shape=(lookback, 1), activation = 'relu',␣
↪return_sequences=False, name='LSTM'))
```

```
    model.add(Dense(units=1, name='Output'))              # can also specify␣
 ↪linear activation function

    # specify optimizer separately (preferred method))
#     opt = RMSprop(lr=0.001, rho=0.9, epsilon=1e-08, decay=0.0)
    opt = Adam(lr=0.001, epsilon=1e-08, decay=0.0)       # adam optimizer seems␣
 ↪to perform better for a single lstm

    # model compilation
    model.compile(optimizer=opt, loss='mse', metrics=['mae'])

    return model
```

[ ]:
```
# lstm network
model = create_model(hu=10, lookback=lookback)
```

[ ]:
```
# summary
model.summary()
```

[ ]:
```
plot_model(model, to_file='./img/model.png', show_shapes=True,␣
 ↪show_layer_names=True)
```

4.6.1 Train the model

[ ]:
```
# Specify callback functions
model_path = (results_path / 'model.h5').as_posix()
logdir = os.path.join("logs", datetime.datetime.now().strftime("%Y%m%d-%H%M%S"))

my_callbacks = [
    EarlyStopping(patience=10, monitor='loss', mode='min', verbose=1,␣
 ↪restore_best_weights=True),
    ModelCheckpoint(filepath=model_path, verbose=1, monitor='loss',␣
 ↪save_best_only=True),
    TensorBoard(log_dir=logdir, histogram_freq=1)
]
```

[ ]:
```
# Model fitting
lstm_training = model.fit(X_train,
                          y_train,
                          batch_size=64,
                          epochs=500,
                          verbose=1,
                          callbacks=my_callbacks,
                          shuffle=False)
```

4.7 Evaluate model performance

[ ]:
```
%tensorboard --logdir logs
```

[ ]:
```
# calculate rmse of loss function
train_rmse_scaled = np.sqrt(model.evaluate(X_train, y_train, verbose=0))
test_rmse_scaled = np.sqrt(model.evaluate(X_test, y_test, verbose=0))
print(f'Train RMSE: {train_rmse_scaled[0]:.4f} | Test RMSE: {test_rmse_scaled[0]:
 ↪.4f}')
```

4.7.1 Prediction

[ ]:
```
# predictions
y_pred = model.predict(X_test)
```

[ ]:
```
df = pd.DataFrame({
    'actual': scaler.inverse_transform(y_test).flatten(),
    'prediction': scaler.inverse_transform(y_pred).flatten()},
    index = test_data[lookback:].index)

df['spread'] = df['prediction'] - df['actual']
df
```

[ ]:
```
print(f'R-square: {r2_score(df.actual, df.prediction):0.4}')
```

4.7.2 Plot results

[ ]:
```
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(20,6))

ax[0].plot(df.actual, color='red', label='actual')
ax[0].plot(df.prediction, color='blue', label='prediction')
ax[1].hist(df.spread, bins=50, density=True, label='spread')

ax[0].legend()
ax[1].legend()

plt.suptitle('SPY LSTM Prediction');
```

5 References

1. Andrej Karpathy (2015), The Unreasonable Effectiveness of Recurrent Neural Networks
2. Kannan Singaravelu (2023), Introduction to Deep Learning & Neural Networks, Slide No.84
3. Keras API
4. TensorFlow API
5. Scikit-Learn Preprocessing
6. Francois Chollet (2019), Deep Learning with Python
7. Stefan Jansen (2020), Machine Learning for Algorithmic Trading
8. Yves Hilpisch (2020), Artificial Intelligence in Finance

June 2023, Certificate in Quantitative Finance.